ALGORITHMIQUE EN BASH · EXERCICES DU JEUDI 22 OCTOBRE

Quatre exercices pour s'entraîner

Quatre petits scripts, un par exercice. Ils réutilisent ce que vous avez déjà fait cette semaine : les tableaux de la moyenne, la boucle lettre par lettre du palindrome et du pendu, les fonctions.

Pour chaque exercice, avancez étape par étape et lancez le script après chaque étape. Votre sortie doit être la même que l'exemple avant de passer à la suite. Les bonus sont pour ceux qui ont fini, pas une obligation.

Exercice 1 · La météo de la semaine

Créez `meteo.sh`. On a relevé la température chaque jour pendant une semaine. Le script doit trouver le jour le plus chaud et le plus froid.

Étape 1. Créez le tableau, puis affichez toutes les températures et le nombre de jours. Comme pour la moyenne, `${temperatures[*]}` donne toutes les valeurs et `${#temperatures[@]}` en donne le nombre.

```
temperatures=(12 15 9 21 18 7 14)
```

Exemple d'exécution

```
$ ./meteo.sh
Températures : 12 15 9 21 18 7 14
Nombre de jours : 7
```

Étape 2 · La plus chaude. On ne connaît pas encore la plus haute température, alors on fait comme si c'était la première de la liste :

```
max=${temperatures[0]}
```

Puis on parcourt le tableau avec une boucle `for t in "${temperatures[@]}"`. À chaque tour, si `t` est plus grand que `max`, on remplace `max` par `t`. Pour comparer deux nombres : `if (( t > max )); then`.

À la fin de la boucle, `max` contient la plus haute température. Affichez-la.

Étape 3 · La plus froide. Même chose avec une variable `min`. Une seule différence : le signe de la comparaison.

Étape 4 · L'écart. Affichez la différence entre la plus chaude et la plus froide, avec `$(( max - min ))`.

Exemple d'exécution

```
$ ./meteo.sh
Températures : 12 15 9 21 18 7 14
Nombre de jours : 7
Plus chaude : 21
Plus froide : 7
Écart : 14
```

Si vous bloquez. Si vous obtenez 14 (la dernière température de la liste), votre `if` est après la boucle au lieu d'être dedans, entre `do` et `done`. Si le script affiche une erreur sur la ligne du `if`, vérifiez les espaces dans `(( t > max ))`.

Bonus. Ajoutez un deuxième tableau `jours=(lundi mardi mercredi jeudi vendredi samedi dimanche)` et affichez **quel jour** il a fait le plus chaud. Astuce : au lieu de garder la température la plus haute, gardez son **indice**. Parcourez le tableau avec `for i in "${!temperatures[@]}"` : le `!` donne les indices 0, 1, 2… au lieu des valeurs. Le même indice `i` sert ensuite pour lire `${jours[i]}`.

Exemple d'exécution

```
$ ./meteo.sh
La plus chaude : 21 degrés, jeudi
```

Exercice 2 · Compter une lettre

Créez `lettres.sh`. Le but est d'écrire une fonction `compter_lettre` qui compte combien de fois une lettre apparaît dans un mot.

Étape 1 · La fonction. Elle reçoit deux arguments : le mot dans `$1`, la lettre dans `$2`. Commencez par les ranger dans des variables locales, et créez un compteur :

```
compter_lettre() {
    local mot=$1
    local lettre=$2
    local compteur=0

    # la boucle ici

    echo "$compteur"
}
```

Pour la boucle, reprenez celle du pendu, qui passe sur chaque indice du mot : `for (( i = 0; i < ${#mot}; i++ ))`. À chaque tour, `${mot:i:1}` est la lettre à l'indice `i`. Si elle est égale à `$lettre`, ajoutez 1 au compteur avec `(( compteur++ ))`.

Pour comparer deux lettres : `if [[ ${mot:i:1} == "$lettre" ]]; then`.

Testez la fonction avec ces trois lignes, en bas du script :

```
compter_lettre "banane" "a"
compter_lettre "algorithme" "o"
compter_lettre "bash" "z"
```

Exemple d'exécution

```
$ ./lettres.sh
2
1
0
```

Étape 2 · Le programme. Remplacez les trois lignes de test. Le mot et la lettre arrivent maintenant en arguments du script : `./lettres.sh banane a`. Rangez le résultat de la fonction dans une variable avec `$( )`, comme pour `inverser` dans le palindrome, puis affichez une phrase complète :

```
nb=$(compter_lettre "$1" "$2")
```

Exemple d'exécution

```
$ ./lettres.sh banane a
« a » apparaît 2 fois dans « banane »
$ ./lettres.sh algorithme z
« z » apparaît 0 fois dans « algorithme »
```

Si vous bloquez. Si le résultat est toujours 0, affichez `${mot:i:1}` dans la boucle pour voir les lettres que le script compare. Si la fonction affiche plusieurs nombres, c'est que le `echo "$compteur"` est dans la boucle au lieu d'être après.

Bonus. Écrivez un deuxième script qui compte **toutes les voyelles** d'un mot. Pas besoin de nouvelle fonction : faites une boucle `for v in a e i o u y`, appelez `compter_lettre` pour chaque voyelle et additionnez les résultats dans un total.

Exemple d'exécution

```
$ ./voyelles.sh algorithme
4 voyelles dans « algorithme »
```

Exercice 3 · Admis ou recalé

Créez `admis.sh`. On reprend un tableau de notes, comme pour la moyenne.

```
notes=(12 8 15 10 17 6 11)
```

Étape 1. Parcourez les notes avec une boucle `for`. Pour chaque note, affichez « admis » si elle vaut 10 ou plus, « recalé » sinon. « 10 ou plus » s'écrit `(( n >= 10 ))`.

Étape 2. Créez une variable `admis=0` **avant** la boucle. Dans la boucle, ajoutez 1 à chaque note admise. Après la boucle, affichez le nombre d'admis sur le nombre total de notes.

Exemple d'exécution

```
$ ./admis.sh
12 : admis
8 : recalé
15 : admis
10 : admis
17 : admis
6 : recalé
11 : admis
5 admis sur 7
```

Si vous bloquez. Si vous obtenez « 1 admis sur 7 », la ligne `admis=0` est dans la boucle et le compteur repart à zéro à chaque tour. Si 10 est recalé, vérifiez que vous avez écrit `>=` et pas `>`.

Bonus. Remplacez « admis » par une mention, avec `if`, `elif` et `else` : Très bien à partir de 16, Bien à partir de 12, Passable à partir de 10, Recalé en dessous. Attention à l'ordre des tests : il faut commencer par la mention la plus haute.

Exemple d'exécution

```
$ ./mentions.sh
12 : Bien
8 : Recalé
15 : Bien
10 : Passable
17 : Très bien
6 : Recalé
11 : Passable
```

Exercice 4 · Chercher un prénom

Créez `liste.sh`. Le prénom à chercher arrive en argument : `./liste.sh chloe`.

```
prenoms=(alice bob chloe david emma)
```

Étape 1. Rangez `$1` dans une variable `cible`. Créez une variable `trouve=0`. Parcourez le tableau **par indices** (`for i in "${!prenoms[@]}"`, comme dans le bonus de la météo). Si `${prenoms[i]}` est égal à `$cible`, met-

tez `trouve=1`, gardez l'indice dans une variable `position`, et sortez de la boucle avec `break`. Inutile de continuer à chercher une fois qu'on a trouvé.

Étape 2. Après la boucle, regardez `trouve` : s'il vaut 1, affichez où le prénom a été trouvé, sinon affichez qu'il n'est pas dans la liste.

Étape 3. Ajoutez un compteur `comparaisons=0` avant la boucle, et ajoutez-lui 1 **au début** de chaque tour. Il dit combien de prénoms le script a dû regarder.

Exemple d'exécution

```
$ ./liste.sh chloe
chloe est dans la liste, à l'indice 2
Comparaisons : 3
$ ./liste.sh alice
alice est dans la liste, à l'indice 0
Comparaisons : 1
$ ./liste.sh zoe
zoe n'est pas dans la liste
Comparaisons : 5
```

Regardez le nombre de comparaisons. Alice est trouvée du premier coup, parce qu'elle est en tête de liste. Pour zoe, le script doit tout lire avant de pouvoir répondre non. Avec 5 prénoms, ça ne change rien. Avec un million de lignes, ça change tout : on verra demain comment chercher beaucoup plus vite dans une liste triée.

Si vous bloquez. Si le script trouve n'importe quel prénom à l'indice 0, même zoe, vous comparez avec `(( ))`. C'est fait pour les nombres. Pour du texte, il faut `[[ ]]`. Si le compteur vaut toujours 5, le `break` manque.

Bonus. Rangez la recherche dans une fonction `est_present` qui répond par son code de retour (`return 0` si trouvé, `return 1` sinon), comme `est_palindrome`. Puis testez plusieurs prénoms d'un coup avec une boucle sur `"$@"`.

Exemple d'exécution

```
$ ./presents.sh bob zoe emma
bob : présent
zoe : absent
emma : présent
```